

ForecastIQ: A High-Performance Multi-Horizon Time-Series Forecasting and Benchmarking Platform

Anuj Meena

Abstract

This report outlines the design, architectural paradigms, and validation metrics of the ForecastIQ sequence forecasting platform. ForecastIQ benchmarks three state-of-the-art architectures: Patch-based Time-Series Transformers (PatchTST), univariate fully-connected residual decomposition stacks (N-BEATS), and Temporal Fusion Transformers (TFT) with dynamic covariates. The platform implements channel-independence and subseries patching, reducing Transformer sequence complexity from quadratic to linear. We present the mathematical formulations of the model layers, the scaling and windowing pipelines, accuracy metrics (MAE, RMSE, sMAPE), and the unified validation outputs on Apple M3 CPU architectures.

I. INTRODUCTION

Time-series forecasting is critical across supply chain demand planning, logistics, and resource allocation. While traditional recurrent networks struggle to model long-term temporal dependencies, modern transformer architectures address these limitations. However, choosing the optimal model requires side-by-side benchmarking.

ForecastIQ provides a standardized environment comparing three distinct forecasting architectures: PatchTST, N-BEATS, and a custom Gated TFT. The system is designed to predict a multi-step horizon of 24 steps into the future based on a historical context window of 96 steps.

II. FORECASTING MODEL ARCHITECTURES

A. PatchTST (*Patch Time-Series Transformer*)

PatchTST implements two key enhancements over standard transformers:

- 1) **Channel Independence:** The input multivariate time-series of shape $[\text{Seq_Len}, M]$ is treated as M independent univariate series. Each channel undergoes the same encoder weights, preventing feature mixing before sequence representation.
- 2) **Subseries Patching:** Instead of feeding single steps to the attention layers, the univariate series is grouped into overlapping patches of length P with stride S . Let $x \in \mathbb{R}^L$ be a univariate series of length L . The patches $x_p \in \mathbb{R}^{P \times N}$ are defined as:

$$x_p = [x_1, x_2, \dots, x_N] \quad (1)$$

where $N = \lfloor (L - P)/S \rfloor + 1$ is the number of patches. This reduces attention complexity from $\mathcal{O}(L^2)$ to $\mathcal{O}(N^2)$, significantly cutting memory footprint.

The patches are projected to a hidden dimension D via a linear map and summed with a positional embedding matrix:

$$z_p = W_{\text{proj}} x_p + W_{\text{pos}} \quad (2)$$

The token matrix z_p is processed by standard Transformer encoder layers.

B. N-BEATS (Neural Basis Expansion Analysis)

N-BEATS is a deep stack of fully-connected blocks utilizing backward and forward residual connections. For block l with input x_l :

$$\theta_l^{\text{back}}, \theta_l^{\text{fore}} = \text{FC}_l(x_l) \quad (3)$$

$$\hat{x}_l = g_l^{\text{back}}(\theta_l^{\text{back}}) \quad (4)$$

$$\hat{y}_l = g_l^{\text{fore}}(\theta_l^{\text{fore}}) \quad (5)$$

where $\hat{x}_l$ is the backcast (reconstruction of input) and $\hat{y}_l$ is the forecast output. The residual connections update inputs for block $l + 1$:

$$x_{l+1} = x_l - \hat{x}_l \quad (6)$$

The final prediction is the sum of forecasts across all blocks: $\hat{y} = \sum_l \hat{y}_l$. We configure blocks into separate trend and seasonality stacks to improve decomposition.

C. Temporal Fusion Transformer (TFT)

TFT integrates self-attention with Gated Residual Networks (GRN) utilizing Gated Linear Units (GLU) to filter out irrelevant static or temporal features:

$$\text{GLU}(x) = \sigma(W_1x + b_1) \otimes (W_2x + b_2) \quad (7)$$

where $\otimes$ is the element-wise product and $\sigma(\cdot)$ is the sigmoid function.

III. SYSTEM ARCHITECTURE & CLASS DIAGRAMS

The internal codebase uses modular object wrappers, separating model declarations from evaluation pipelines.

A. Data Sequence Flows

The time-series forecasting data pipeline follows a structured sequence:

- 1) ****Data Ingestion****: Ingests historical multi-frequency data via CSV format.
- 2) ****Pre-processing Block****: Computes min-max scaling to project inputs to $[0, 1]$.
- 3) ****Sliding Window Generator****: Splits the normalized series into sliding context blocks ($L = 96$ steps) and horizon target blocks ($H = 24$ steps).
- 4) ****Model Forecast****: Feeds the context blocks into the active forecasting models.
- 5) ****Scale Inversion****: Inverts predictions to calculate actual scaling bounds.
- 6) ****Validation Engine****: Computes error metrics and outputs visual forecasting results.

B. TimeSeriesProcessor Class Map

The processor module handles indexing and normalization transforms:

```

+-----+
|                                     |
|               TimeSeriesProcessor |
| - data: DataFrame                   |
| - scaler: MinMaxScaler              |
+-----+
| + __init__(file_path: str)         |
| + load_and_scale() -> ndarray      |
| + create_windows(context_len: int, horizon_len: int) |
| + inverse_scale(predictions: Tensor) -> Tensor      |
+-----+
|                                     |
|                                     |
|                                     |
+-----+

```

```

|                                     EvaluationEngine                                     |
+-----+
| + calculate_metrics(y_true, y_pred) -> dict                                     |
| + plot_forecasts(y_true, y_pred, model_name: str)                             |
+-----+

```

IV. MODEL HYPERPARAMETERS & TENSOR DIMENSIONS

To guarantee full reproducibility, the model dimensions are specified as follows:

- **Sliding Window Bounds:**
 - Context Window Length (L): 96 steps.
 - Forecast Horizon (H): 24 steps.
- **PatchTST Network Dims:**
 - Input Tensor Shape: $[B, 96, 1]$ where B is the batch size (default = 16).
 - Patch Length (P): 16 steps.
 - Stride (S): 8 steps.
 - Resulting Token Count (N): 11 patches.
 - Feature Embeddings Channel (D): 128 channels.
 - Attention Encoder Heads: 4.
- **N-BEATS Stack Dims:**
 - Input Tensor Shape: $[B, 96]$
 - Trend Blocks: 3, seasonality blocks: 3.
 - Inner Linear Layer Dimension: 512.
 - Target Forecast Shape: $[B, 24]$
- **Optimizer Config:** Adam optimizer with learning rate $\eta = 5e - 4$, decay = $1e - 6$, trained over 10 epochs.

V. METRIC FORMULATION

Three key metrics are calculated to evaluate accuracy:

$$\text{MAE} = \frac{1}{M \cdot H} \sum_{i=1}^M \sum_{t=1}^H |y_{i,t} - \hat{y}_{i,t}| \quad (8)$$

$$\text{RMSE} = \sqrt{\frac{1}{M \cdot H} \sum_{i=1}^M \sum_{t=1}^H (y_{i,t} - \hat{y}_{i,t})^2} \quad (9)$$

$$\text{sMAPE} = \frac{100\%}{M \cdot H} \sum_{i=1}^M \sum_{t=1}^H \frac{|y_{i,t} - \hat{y}_{i,t}|}{(|y_{i,t}| + |\hat{y}_{i,t}|)/2} \quad (10)$$

VI. PERFORMANCE BENCHMARKS

Benchmarks were run on a single-threaded CPU environment (Image size = None, Batch Size = 16, Context Length = 96).

TABLE I
INFERENCE LATENCY COMPARISONS

Model Architecture	Device	Batch Latency (ms)	Throughput (Batches/sec)
PatchTST	CPU	0.53 ms	1899.12
N-BEATS	CPU	0.40 ms	2492.23

VII. DEVOPS & VERIFICATION

ForecastIQ implements the AI Engineering Verification Framework. Running ‘make verify’ executes Ruff formatting, Mypy checks, unit tests, and compiles the JSON reports under ‘reports/latest.json’. The codebase features 19 modules, 8 unit tests, FastAPI endpoints, a Streamlit dashboard, and Docker configurations.